

System and Devices (SYS 3)

Lecture 14: Page Replacement

Dr Pengcheng Liu

Recap

Complexity



Single user contiguous memory

Fixed partitions

Dynamic partitions

Relocatable dynamic partitions

Paged memory allocation

Demand Paging

Segmented memory

Segmented + demand paging = Virtual memory

Recap

- Virtual memory allows separation of logical and physical address spaces
 - Only part of the program must be loaded
 - Logical address space partitioned in fixed-sized pages
 - Memory partitioned in fixed-sized frames
- Demand paging loads fixed-sized blocks from backing store into memory frames when needed
 - If no free frames available, page replacement needed
 - Replacement policies are crucial to system performance

What are we going to learn today?

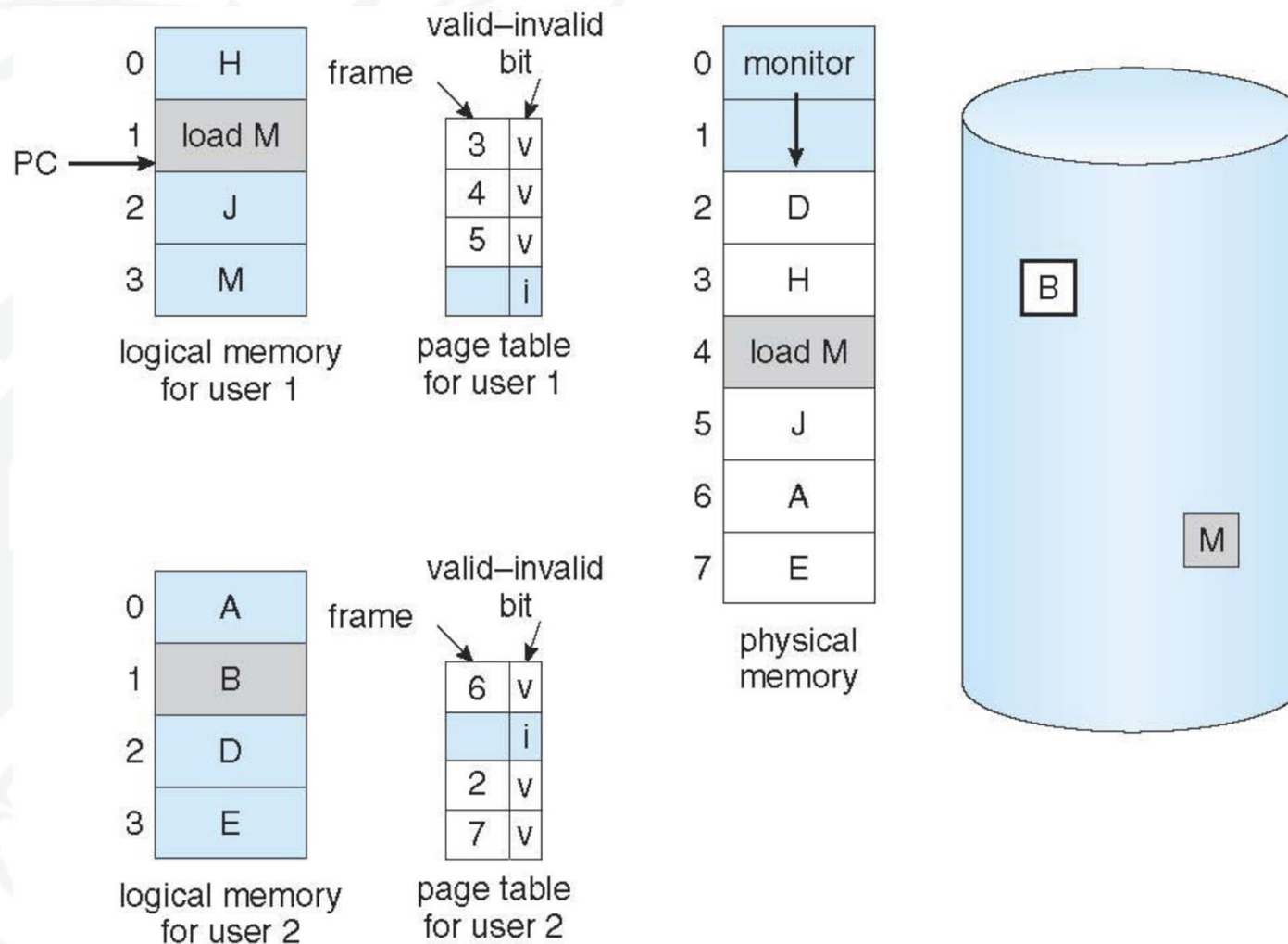
- Page Replacement
- Policies
 - FIFO, OPT, LRU and approximations
- Thrashing
- Working Set Model

Page Replacement

- Page replacement occurs when
 - A page fault occurs and we need to bring the desired page into memory
 - There are NO free frames.
- Page replacement –find some page in memory, but not really in use, page it out
 - Algorithm –decide which frame to free
 - Performance –want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

Page Replacement -Example

All frames are used. No free frames. User 2 needs B



Page Replacement

- Optimisation: use modify (dirty) bit to reduce overhead of page transfers
 - Only modified pages are written back to disk
- Page replacement completes separation between logical memory and physical memory
 - Large virtual memory can be provided on a smaller physical memory

Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame
 - if there is a free frame, use it
 - if there is no free frame, use a page replacement algorithm
 - select a victim frame
 - write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

Page Replacement

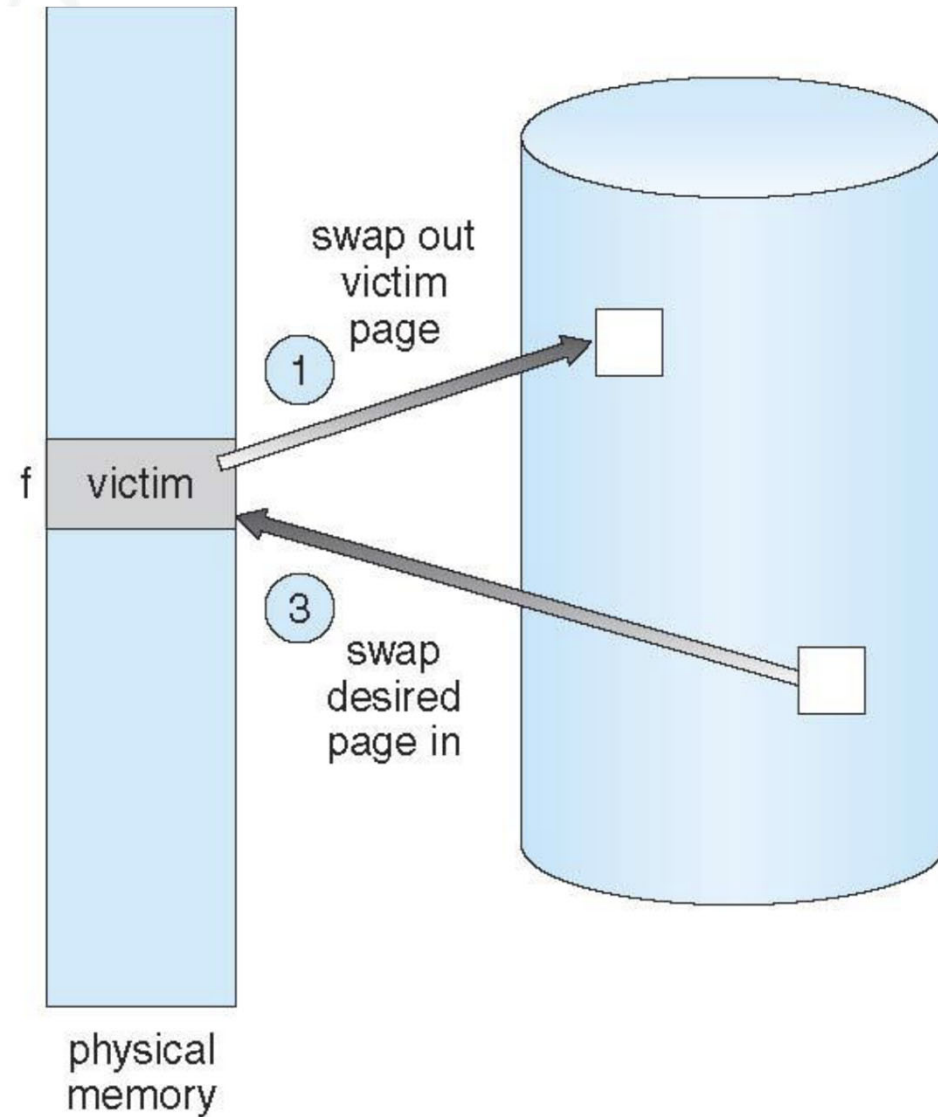
frame valid-invalid bit

0	i
f	v

page table

② change to invalid

④ reset page table for new page



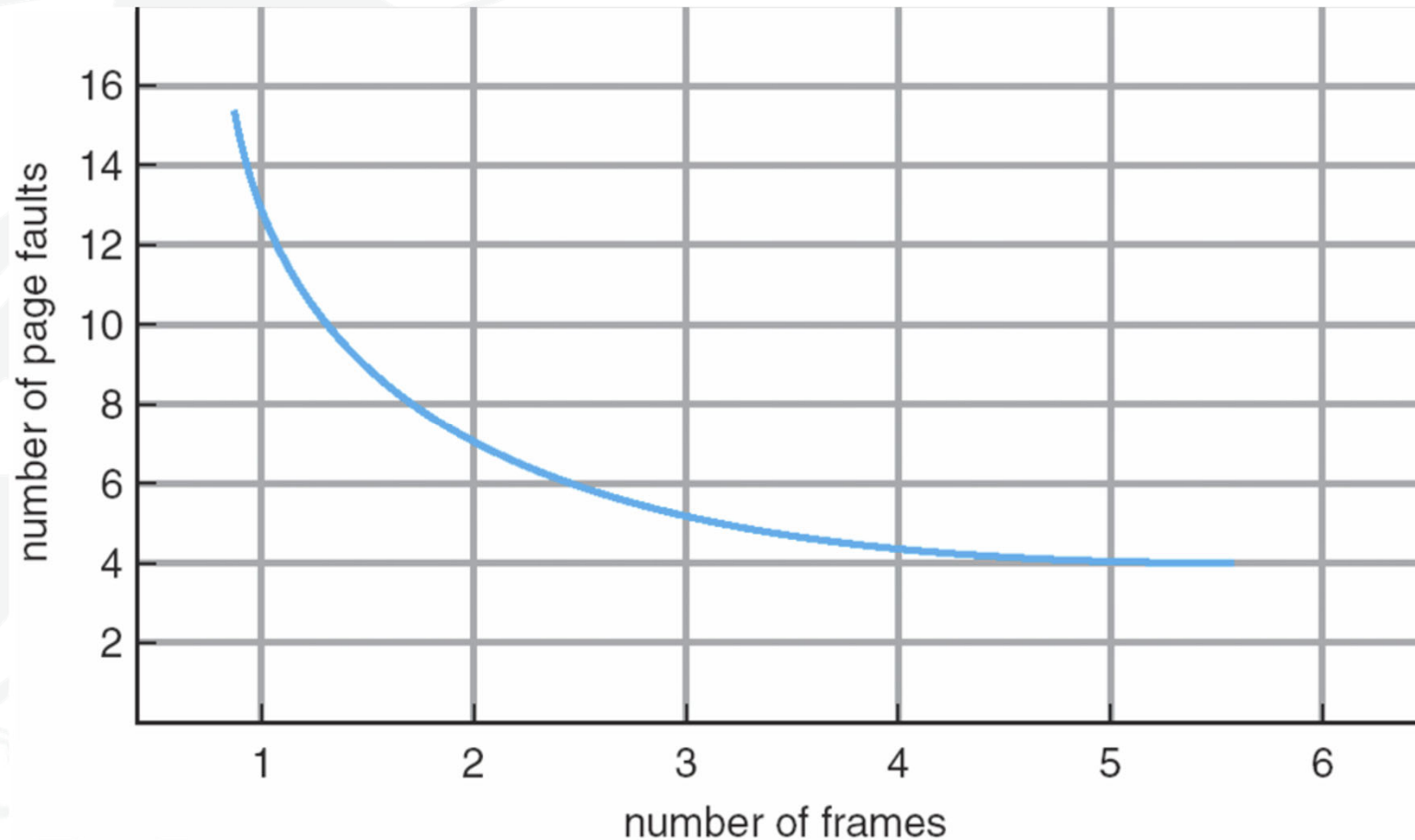
Page Replacement Algorithms

- Page-replacement algorithm aims for lowest page-fault rate on both first access and re-access
- Algorithm evaluation by running it on a particular string of memory references and computing the number of page faults on that string
 - String is just page numbers, not full addresses
 - Repeated access to the same page does not cause a page fault
 - Results depend on number of frames available
- In all our examples, the **reference string** of referenced page numbers is

7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

Page Replacement

- Typical relation between # frames and # page faults



Page Replacement Algorithms -FIFO

- Reference string:
7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1
- 3 frames (3 pages can be in memory at a time per process)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2					2	2	4	4	4	0					0	0			7	7	7
	0	0	0					3	3	3	2	2	2					1	1			1	0	0
		1	1					1	0	0	0	3	3					3	2			2	2	1

page frames

15 page faults

- How to track ages of pages?
- Just use a FIFO queue

Page Replacement Algorithms -FIFO

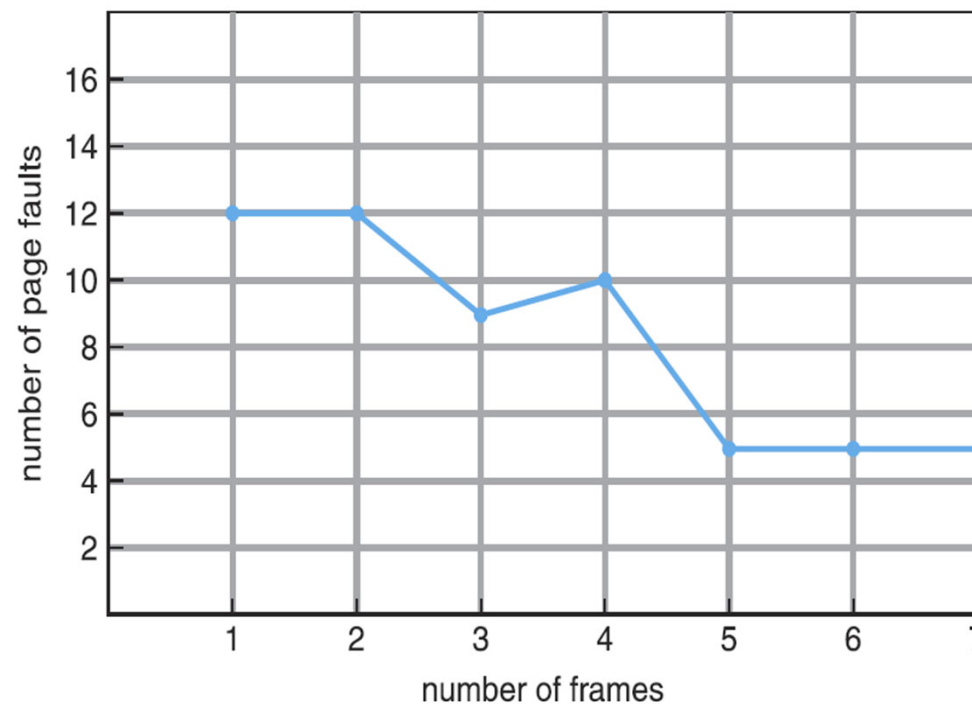


UNIVERSITY
of York

- One would expect that the more frames are allocated to a process the fewer page faults
- Consider the reference string and FIFO replacement algorithm
1,2,3,4,1,2,5,1,2,3,4,5
- How many page faults if we have 3 frames?
- How many page faults if we have 4 frames?

Page Replacement Algorithms -FIFO

- If we use FIFO for page replacement we can encounter the anomaly that more frames may lead to more page faults
- This was first studied by Hungarian computer scientist László Bélády, and is now known as Belady's Anomaly

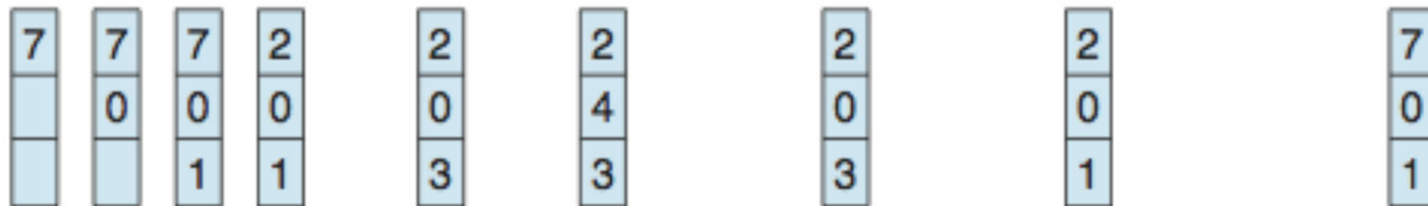


Page Replacement Algorithms - Optimal

- Best possible replacement policy: replace page that will not be used for longest period of time
 - With 3 frames, 9 is optimal for the example reference

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

- How to find which page will not be used for longest period of time
 - Only theoretical, can't read the future
 - Used mainly as upper-bound in comparative evaluation

Page Replacement Algorithms -LRU

- Least Recently Used (LRU)
 - Use past knowledge rather than future: replace page that has not been used for the longest period of time
 - Must keep track of time of last use for each page

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1
	0	0	0		0		0	0	3	3		3		0		0
		1	1		3		3	2	2	2		2		2		7

page frames

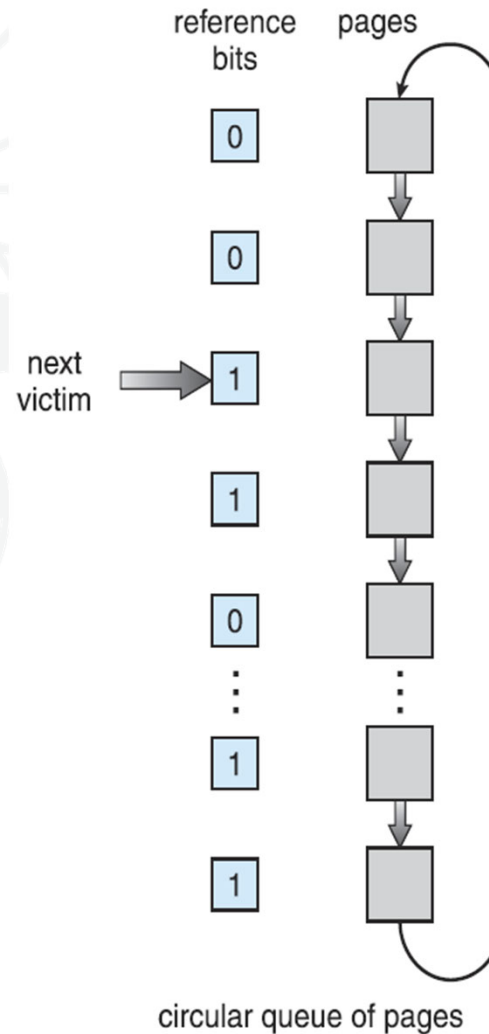
- 12 faults, better than FIFO but worst than OPT
- Generally good and widely used, but there are implementation issues

LRU Approximation Algorithms

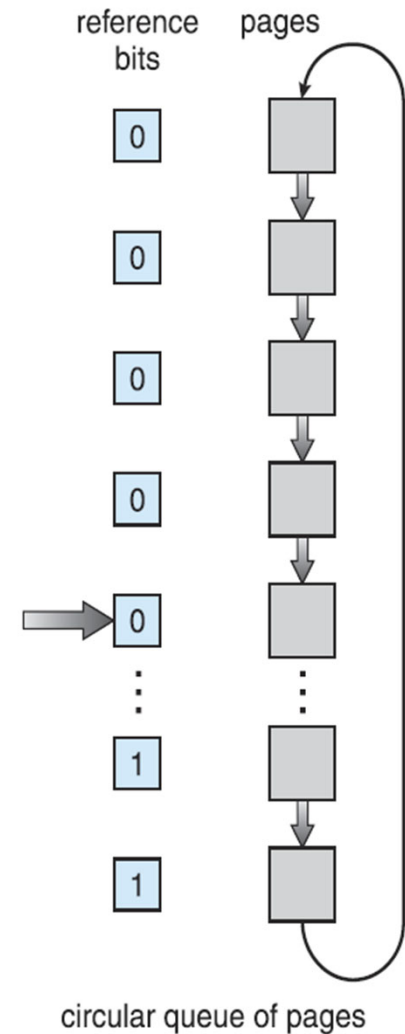
- Similar to LRU, but without complex timestamping
 - **Reference bit**
 - with each page associate a hardware-provided bit; initially = 0
 - when a page is referenced the associated bit is set to 1
 - replace any page with reference bit = 0 (if one exists)
 - we do not know the order, however
- **Second-chance algorithm**
 - FIFO scheme, plus hardware-provided reference bit
 - if page to be replaced has
 - reference bit = 0 → replace it
 - reference bit = 1 then:
 - set reference bit 0, leave page in memory
 - replace next page, subject to same rules

Second Chance Algorithm

Next victim is a page with reference bit 1, so move on to first page with reference bit 0, changing all reference bits that are currently 1 to 0 on the way



(a)



(b)

Page and Frame Locking

- OS may wish some pages to remain in physical memory frames
 - Parts of the OS code itself
 - I/O buffers
- Frame locking
 - If frame is locked, it may not be replaced
 - Requires a lock bit with each frame
 - usually supported in OS rather than hardware

Frame Allocation

- **Frame allocation algorithm** determines how many frames to give each process
 - and which frames to replace in case of scarcity
- Each process needs **minimum** number of frames
 - example: IBM 370 –6 pages to handle SS MOVE instruction
 - instruction is 6 bytes, might span 2 pages
 - 2 pages to handle from
 - 2 pages to handle to
- **Maximum** of course is total frames in the system
- Many allocation schemes
 - Equal allocation, size-proportional allocation, priority allocation, etc.
 - Important to keep a free frame buffer pool

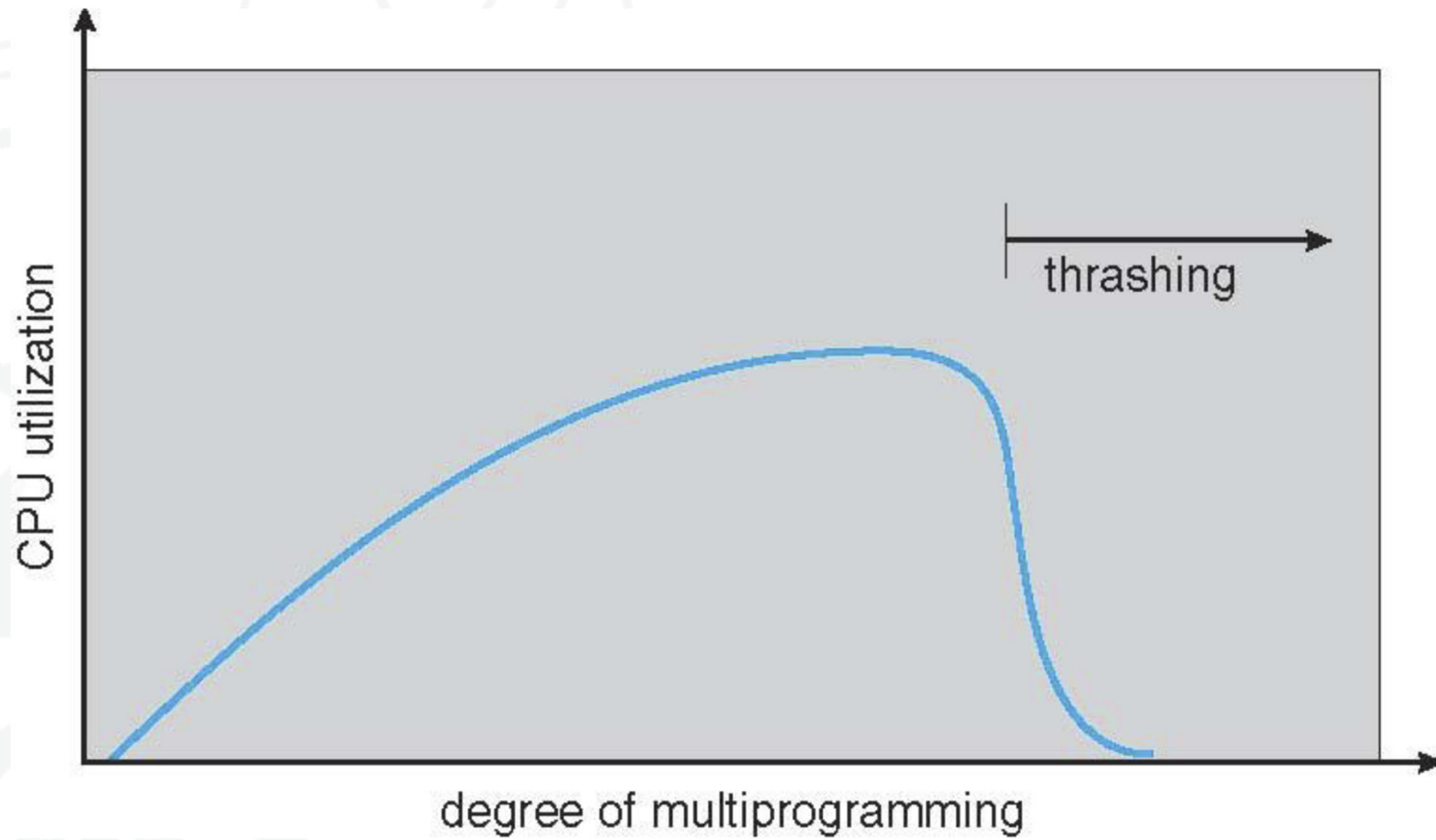
Frame Allocation

- Global replacement—process selects a replacement frame from the set of all frames
 - One process can take a frame from another
 - In priority based allocation of frames, this may enable a high priority process to increase its allocated frames by taking a frame from a low priority process
 - Page fault behaviour of a process becomes dependent on the behaviour of other processes
 - Greater overall throughput, so more common (e.g. Linux)
- Local replacement—each process selects from only its own set of allocated frames
 - More consistent per-process performance
 - If a process does not have sufficient number of frames allocated to it, the process will suffer many page faults (thrashing)
 - Possibly underutilized memory

Thrashing

- If a process does not have “enough” pages, the page-fault rate is very high
 - Insufficient frames leads to page fault
 - No free frames (otherwise they would have been allocated to the faulting process) hence have to swap out a frame of another (active) process
 - and allocate the frame to the faulting process
 - Page swapped out is needed again, so page fault, no free frames
 - This can lead to
 - low CPU utilization
 - operating system thinking that it needs to increase the degree of multiprogramming
 - another process added to the system
- Thrashing: a process is busy swapping pages in and out

Thrashing



Thrashing

- Why does demand paging work?

Locality model

- Process migrates from one locality to another
- Localities may overlap
- Why does thrashing occur?

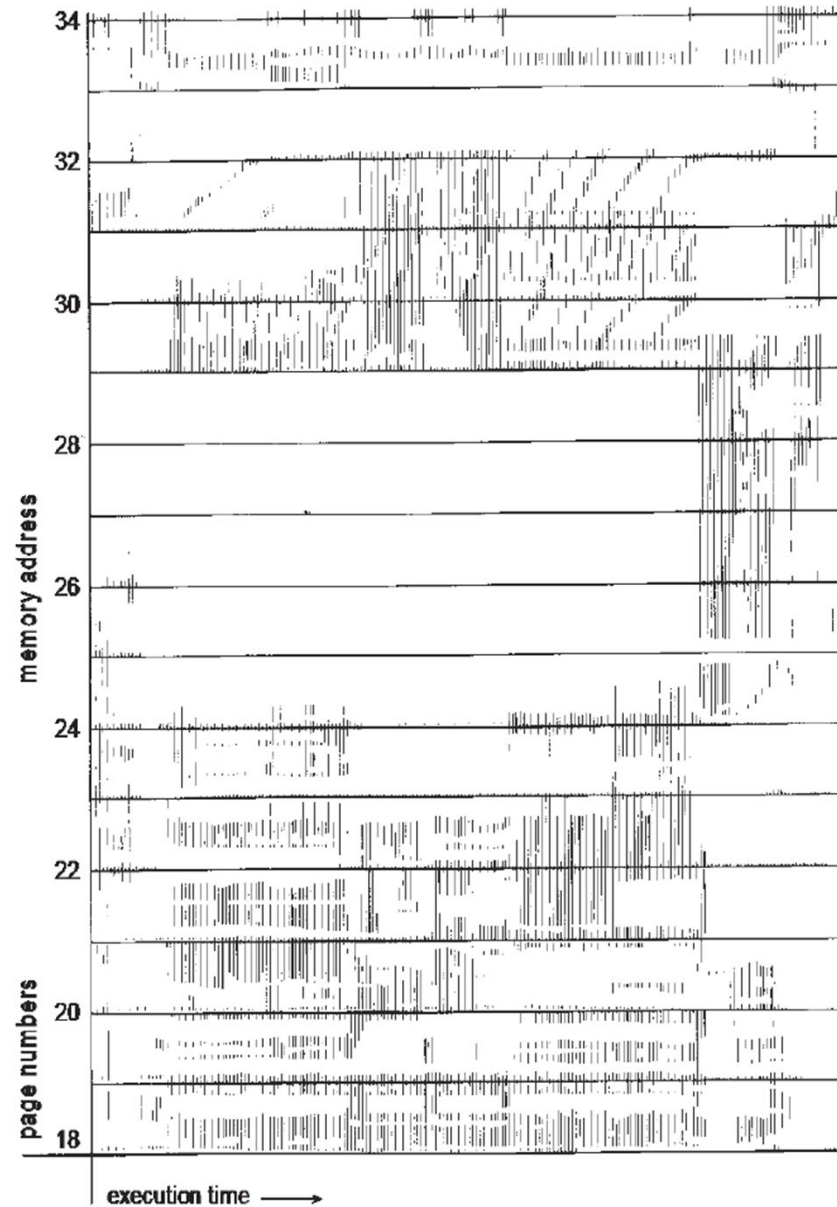
Σ size of locality > total memory size

- Can limit the effects of thrashing by using:
 - local page replacement
 - priority page replacement –replace a page from a process with the lowest priority.

Locality - Example



UNIVERSITY
of York

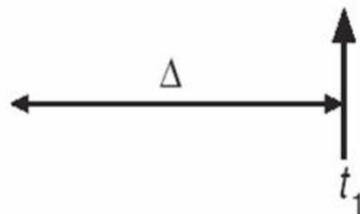


Working-Set Model

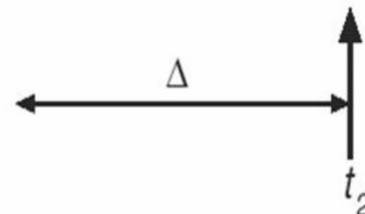
- Define Δ to be a working-set window
 - Analyse the most recent Δ page references
 - If a page is in use, it is in the working set Δ
 - If it is no longer used, it will drop from the working set Δ time units after its last reference
- WSS_i (working set of Process P_i) is defined to be the total number of pages referenced in the window Δ
 - Example with $\Delta = 10$

page reference table

... 2 6 1 5 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...



$$WS(t_1) = \{1, 2, 5, 6, 7\}$$



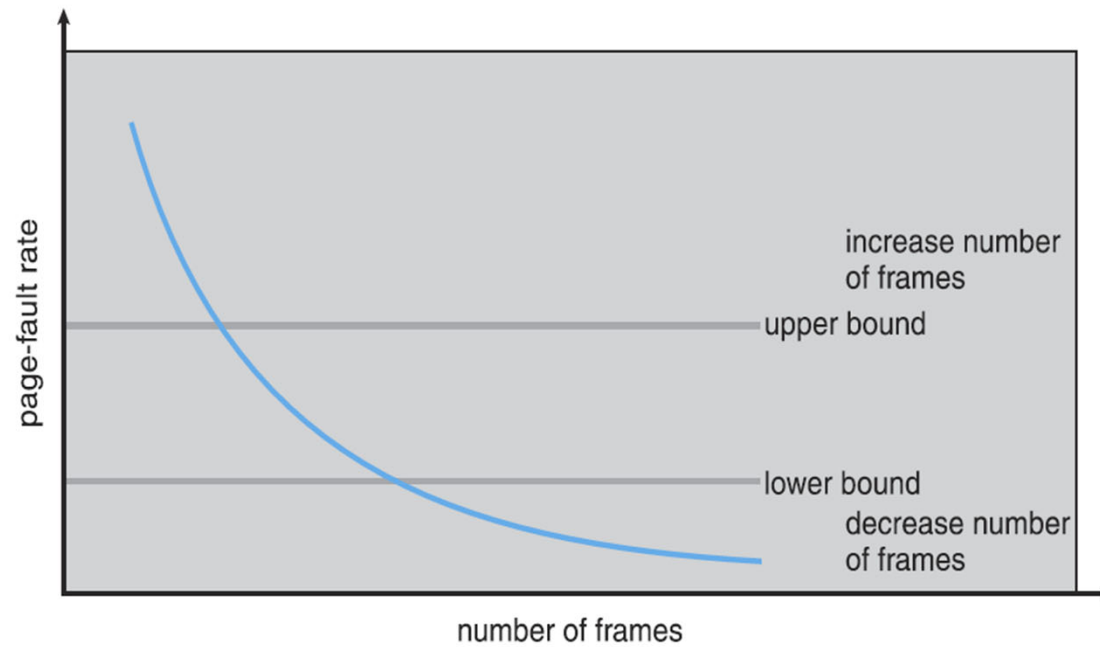
$$WS(t_2) = \{3, 4\}$$

Working-Set Model

- WSS_i tries to approximate the size of the locality of process P_i
 - If Δ too small will not encompass entire locality
 - If Δ too large will encompass several localities
 - If $\Delta = \infty \Rightarrow$ will encompass entire program
- $D = \sum_{i=0}^n WSS_i$
 - Total demand for frames by all processes $P_0 \dots P_n$
- If $D > m \Rightarrow$ Thrashing
 - m = total number of frames
- Policy: if $D > m$, then suspend or swap out one of the processes

Page-Fault Frequency

- More direct approach than WSS
- Establish “acceptable” page-fault frequency (PFF) rate and use local replacement policy
 - If actual rate too low, process loses frame
 - If actual rate too high, process gains frame



Suggested Reading

- Silberschatz, chapter 10, 10th Edition